



SECURITY REVIEW • PREPARED FOR

Hood Domains

An ENSv2-based naming service on
Robinhood Chain

DATE

16.09.2026

COMMIT

6e5818b9

REMEDiation

8a87c91d

Phase & process

1. About SBSecurity

SBSecurity is a team of skilled smart contract security researchers. Based on the audits conducted and numerous vulnerabilities reported, we strive to provide the absolute best security service and client satisfaction. While it's understood that 100% security and bug-free code cannot be guaranteed by anyone, we are committed to giving our utmost to provide the best possible outcome for you and your product.

Book a Security Review with us at sbsecurity.net or reach out on Telegram [@sbsecurity_audits](https://t.me/sbsecurity_audits).

2. Disclaimer

A smart contract security review can only show the presence of vulnerabilities **but not their absence**. Audits are a time, resource, and expertise-bound effort where skilled technicians evaluate the codebase and their dependencies using various techniques to find as many flaws as possible and suggest security related improvements. We as a company stand behind our brand and the level of service that is provided but also recommend subsequent security reviews, on-chain monitoring, and high whitehat incentivization.

3. Risk classification

	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

3.1 IMPACT

- **High** - leads to a significant loss of assets in the protocol or significantly harms a group of users.
- **Medium** - leads to a moderate loss of assets in the protocol or some disruption of the protocol's functionality.
- **Low** - funds are not at risk.

3.2 LIKELIHOOD

- **High** - almost certain to happen, easy to perform, or highly incentivized.
- **Medium** - only conditionally possible, but still relatively likely.
- **Low** - requires specific state or little-to-no incentive.

02 EXECUTIVE SUMMARY

Key findings

TOTAL FINDINGS

9

RESOLVED

5 of 9 · 56%

HIGH SEVERITY FINDINGS

0 none found

SEVERITY DISTRIBUTION

☒ Fixed & verified ☒ Acknowledged

CRITICAL 0

HIGH 0

MEDIUM 0

LOW 3 3

INFORMATIONAL 2 1

GAS OPT. 0

CLIENT

Hood Domains

REPOSITORY

Private

METHODS

Manual Review,
AI Review

TIMELINE · SEPTEMBER 2026

Audit · 5 days

Remediation · 1 day

KICK-OFF	END OF AUDIT	REMEDiations START	REMEDiations END
10.09.2026	16.09.2026	16.09.2026	16.09.2026

STARTING COMMIT HASH 6e5818b90b4f85835a61574bf908e229a34c8083

FINAL COMMIT HASH 8a87c91da32481735d06e49fae53abfc829e7dc0

CONTRACTS IN SCOPE

```
src/registrar/AbstractHoodRegistrar.sol
src/registrar/HoodRegistrarBatch.sol
src/registrar/libraries/LibHalving.sol
src/registry/HoodURIRenderer.sol

src/registry/PermissionedRegistry.sol
(diff only)
src/access-control/libraries/
EACBaseRolesLib.sol (diff only)
src/erc1155/ERC1155Singleton.sol
(diff only)
src/reverse-registrar/
StandaloneReverseRegistrar.sol (diff only)
script/deploy-constants.ts
deploy/01_HoodRegistry.ts
deploy/03_HoodRegistrar.ts
deploy/05_HoodURIRenderer.ts

src/registrar/HoodRegistrar.sol
src/registrar/EthRentPriceOracle.sol
src/registry/HoodRegistry.sol
src/reverse-registrar/
DefaultReverseRegistrar.sol
src/access-control/
EnhancedAccessControl.sol (diff only)
src/registry/libraries/RegistryRolesLib.sol
(diff only)
src/registrar/BatchRegistrar.sol
(diff only)
src/utils/LabelStore.sol

deploy/00_RootRegistry.ts
deploy/02_ReverseRegistry.ts
deploy/04_BatchRegistrar.ts
```

03 PROJECT SUMMARY

What Hood Domains is

Hood Domains is the .hood naming service on Robinhood Chain. Names are ERC-1155 tokens held in a permissioned registry, and a registrar sells and renews them for native ETH at a price set by an on-chain oracle. The price depends on the label length and the duration, and a decaying premium applies to names that expired recently.

The contracts are a fork of ENSv2. The .hood contracts were reviewed in full. For the ENSv2 contracts that changed after ffe4a73, the last commit audited by ENS, only the diff against that commit was reviewed. They are marked (diff only) in the scope list, and the unchanged code in those files was read for context and is treated as audited by ENS. LabelStore.sol is new since that commit and was reviewed in full. The deployment configuration was checked against the live contracts.

SECURITY POSTURE

BEFORE REMEDIATION

6E5818B9

AFTER REMEDIATION

8A87C91D

Registry roles on the live deployment

L-03 · L-04 · L-05

The deploy scripts granted root roles and never revoked them. The superseded HoodRegistrar stayed a live entry point, the deployer key held the root admin roles instead of the documented owner, and no account could complete a reserved name.

Pricing and payments

L-01 · L-02 · I-01 · I-02

Rounding in the halving math could revert quotes late in the premium window, and the oracle accepted a zero premium period. Zero-width characters could move a short name into a cheaper tier, and payments used a plain call that copied return data it never read.

Operator approvals and role changes

L-06 · I-03

Both sit in registry code inherited from ENSv2. An approved operator could revoke the owner's admin roles and leave the name impossible to transfer, and an owner contract that rejected ERC-1155 tokens blocked its own role changes.

Five of nine findings are fixed and four are acknowledged. At the final commit the oracle clamps the premium at zero and rejects a zero premium period, and the registrar sends ETH through an assembly call that copies no return data.

On chain, the team revoked the root roles of the superseded HoodRegistrar and of BatchRegistrar, which resolves L-03 and L-05. We reviewed each code fix at the final commit and confirmed both revocations on the live registry.

The team reviewed the four acknowledged findings and accepted each as a design or operational decision. Two of them follow behaviour that ENSv2 defines upstream.

04 RECOMMENDATIONS

Beyond this audit

Measures that extend the protection of this review after launch, independent of any individual finding.

R-01

Keep privileged keys on a multisig

Registry admin roles and registrar ownership are the highest-value keys in the system. Keep them on a multisig or hardware signers, and when a role moves, grant the new holder before revoking the old one so no role is left without a holder.

R-02

Script every registrar and oracle swap

Registrars and oracles are replaced, not upgraded. Do each swap through one reviewed script that grants the new contract, revokes the old one and records the address, constructor arguments and transaction under deployments/.

R-03

Monitor role and oracle changes

Watch `EACRolesChanged` at the registry root and `RentPriceOracleUpdated` on the registrar, and alert on any change the team did not plan. Privileged changes are rare, so every event deserves a look.

R-04

Normalize names in the application layer

Apply ENSIP-15 normalization in the frontend and the SDK before quoting or registering a label, and show the normalized name to the user before payment.

R-05

Follow ENSv2 upstream

The registry builds on ENSv2. Track upstream releases, fixes and audits, and carry the relevant changes into future registry versions.

Core contracts

CONTRACT	ROLE
HoodRegistry.sol	The .hood registry. A <code>PermissionedRegistry</code> with the collection name, symbol and contract URI added.
PermissionedRegistry.sol	ENSv2 core registry. Names are ERC-1155 tokens with an expiry, per-name roles, and resolver and subregistry pointers.
EnhancedAccessControl.sol	Role bitmaps per resource, each role with a regular and an admin variant, plus the grant and revoke checks behind every permissioned call.
ERC1155Singleton.sol	ERC-1155 with exactly one owner per id, approvals and transfer hooks.
HoodRegistrarBatch.sol	Extends <code>HoodRegistrar</code> with <code>registerBatch</code> , which registers up to 20 names to one owner in a single paid call.
HoodRegistrar.sol	Commit-reveal registrar: commit, then register paid in native ETH, with a minimum duration and a grace period for renewals.
AbstractHoodRegistrar.sol	Registrar base. Holds the registry, the beneficiary and the price oracle, implements renew, forwards the payment and refunds any excess.
EthRentPriceOracle.sol	Prices names in ETH: a base rate per label length, discounts for longer durations, and a decaying premium on recently expired names.

CONTRACT

ROLE

`LibHalving.sol`

Fixed-point halving math behind the premium decay.

`BatchRegistrar.sol`

Owner-only batch reserve and renew, used before the migration.

`HoodURIRenderer.sol`

Builds the token metadata, JSON and SVG, fully on chain. The registry calls it from `uri`.

`LabelStore.sol`

Write-once store of label strings by label hash. The registry calls it on every registration, and it carries the label size check.

`DefaultReverseRegistrar.sol`

Primary names under `default.reverse`. An account sets its own name, and `setNameForAddr` is gated by `AccountNamerLib`.

06 CONTRACTS & ROLES

Privileged roles

Registry admin.

Holds the admin variants of the registrar, reserved registration, renew, parent, URI and naming roles at the root of HoodRegistry, and grants or revokes those roles.

Registrar owner and beneficiary.

Owens the registrar and sets its price oracle through `setRentPriceOracle`. Receives registration and renewal payments as the beneficiary fixed at deployment.

Registrar.

`HoodRegistrarBatch` holds `ROLE_REGISTRAR` and `ROLE_RENEW` at the registry root and is the entry point for paid registrations and renewals.

Name owner.

Set per name at registration. Receives the resolver and subregistry roles with their admin variants, plus the transfer admin role that the registry requires for transfers.

Approved operator.

Any address the owner approves through `setApprovalForAll`. It acts on the owner's names with the owner's roles, as ENSv2 defines.

07 FINDINGS

What we found

ID	TITLE	SEVERITY	STATUS
L-01	Premium rounding can revert quotes and registration before the premium window ends	LOW	● FIXED
L-02	Zero-Width Character Pricing Manipulation	LOW	● ACKNOWLEDGED
L-03	Superseded HoodRegistrar still holds root registrar and renew roles	LOW	● FIXED
L-04	Deployer key holds every root admin role on HoodRegistry	LOW	● ACKNOWLEDGED
L-05	Reserved names cannot be claimed through the live registrars	LOW	● FIXED
L-06	A rejecting owner contract blocks role changes on its own name	LOW	● ACKNOWLEDGED
I-01	Use assembly call to send native tokens when return data is not needed	INFORMATIONAL	● FIXED
I-02	Missing Validation for Premium Period Parameters	INFORMATIONAL	● FIXED
I-03	Approved ERC-1155 operator can revoke the owner's admin roles and permanently lock the name	INFORMATIONAL	● ACKNOWLEDGED

● LOW SEVERITY · 6 FINDINGS

L-01

LOW

● FIXED

Premium rounding can revert quotes and registration before the premium window ends

`getPremiumPriceAfter()` subtracts the premium window's precomputed endpoint value from the current decayed value. This assumes the integer implementation of the decay never falls below its endpoint before the window ends.

`LibHalving._addFraction()` performs a separate rounding-down division for each set fractional bit. Once the remaining amount is small, the accumulated rounding can violate that assumption: a time with many fractional bits can produce a lower result than a later time with fewer bits.

A constructor-accepted example uses a 100 ETH initial premium, a one-day halving period, and a 64-day premium window:

Elapsed time	Raw halving result
63 days	10 wei
64 days - 1 second	0 wei
64 days	5 wei

The stored offset is therefore 5 wei. At 64 days minus one second, the oracle attempts `0 - 5` and reverts with arithmetic panic `0x11`. At 64 days, its window check returns zero without subtraction.

IMPACT

With an affected pricing schedule, a released name is reported as available but its quote and registration revert at affected times late in the premium window. Both public registration paths depend on this oracle method. This is temporary registration unavailability, not theft or an authorization bypass. A user cannot change the oracle schedule; the issue requires an owner-selected schedule that reaches sufficiently small residual amounts.

RECOMMENDATION

Clamp the final subtraction at zero so accumulated rounding effects cannot prevent registration:

```
uint256 current = LibHalving.halving(
    PREMIUM_PRICE_INITIAL, PREMIUM_HALVING_PERIOD, duration
);
return current > PREMIUM_PRICE_OFFSET ? current - PREMIUM_PRICE_OFFSET : 0;
```

Preserve the existing `duration ≥ PREMIUM_PERIOD` early return. Separately, retaining precision until the final rounding step would reduce the numerical discontinuities. Add regression coverage for small residual premiums and fractional-bit boundaries.

L-02

LOW

● ACKNOWLEDGED

Zero-Width Character Pricing Manipulation

`EthRentPriceOracle.getLength()` uses `StringUtils.strlen()` to count Unicode codepoints for tier-based pricing. A user can inject invisible zero-width characters into a label to inflate the codepoint count without adding visible length, qualifying for a cheaper pricing tier.

```
// EthRentPriceOracle.sol

/// @notice Number of Unicode codepoints in `label`.
/// @param label The name to measure.
/// @return The codepoint count.
function getLength(string calldata label) public pure returns (uint256) {
    return StringUtils.strlen(label);
}
```

The base price calculation uses this codepoint count to select a per-second rate from `_baseRatePerCp` :

```
// EthRentPriceOracle.sol

function getBasePrice(string calldata label, uint64 duration) public view returns (uint256) {
    uint256 n = bytes(label).length;
    if (n == 0 || n > 255) {
        return 0; // too long or too short
    }
    uint256 i = getLength(label);
    if (i > _baseRatePerCp.length) {
        i = _baseRatePerCp.length;
    }
    return applyDiscount(_baseRatePerCp[i - 1] * duration, duration);
}
```

IMPACT

A user registering a short name with zero-width characters pays the rate for a longer name. With the deployed rate schedule (0.020 ETH/yr for 1-char, 0.014 ETH/yr for 3-char), a 1-character label padded to 3 codepoints pays ~30% less. The protocol receives less revenue than intended for short names.

RECOMMENDATION

Filter zero-width characters in `_requireAvailable()` or `getBasePrice()`, similar to how `_requireNoUppercase()` already filters ASCII uppercase.

L-03

LOW

● FIXED

Superseded HoodRegistrar still holds root registrar and renew roles

The deploy scripts grant `ROLE_REGISTRAR | ROLE_RENEW` on the `HoodRegistry` root to every registrar they deploy. No script ever revokes a role.

https://github.com/SB-Security/audit-2026-09-HoodDomains/blob/master/hood-contracts/contracts/deploy/03_HoodRegistrar.ts#L40-L49

```
if (!tags.deferV2Registrar) {
  await write(hoodRegistry, {
    functionName: "grantRootRoles",
    args: [DEPLOYMENT_ROLES.HOOD_REGISTRAR_ROOT, hoodRegistrar.address],
    account: deployer,
  });
}
```

The registry accepts `register` and `renew` from any root role holder. Each registrar has its own `owner`, `rentPriceOracle` and `immutables`.

<https://github.com/SB-Security/audit-2026-09-HoodDomains/blob/master/hood-contracts/contracts/src/registrar/AbstractHoodRegistrar.sol#L79-L82>

```
function setRentPriceOracle(IEthRentPriceOracle oracle) external onlyOwner {
  rentPriceOracle = oracle;
  emit RentPriceOracleUpdated(oracle);
}
```

Checked on Robinhood Chain on 2026-09-11 with `roles` on the live registry. All three registrars hold `ROLE_REGISTRAR | ROLE_RENEW` at root:

- `HoodRegistrar` (superseded): `0x59ac59d6a06689cc83b68dab07261cb25b8ca03e`
- `HoodRegistrarBatch` (live): `0xef136bd95cde0cfe2438f6f920ae60cc275c8fa0`
- `BatchRegistrar` : `0x137f98d0ba29ccee94a103542f5e1c732c9e8fc5`

No revocation event was ever emitted at the root resource. The live `HoodRegistrarBatch` has no deploy script and no record under `deployments/`.

IMPACT

The old registrar is a second live entry point. Today both registrars point to the same oracle, owner and beneficiary and use the same 28 day grace period, so users get the same terms on either one. The only difference is `MIN_COMMITMENT_AGE` (60 on the old one, 0 on the live one).

If the owner later changes the oracle on the live registrar only, anyone can keep registering and renewing at the old price through the unrevoked contract. This needs the trusted owner to update one contract and forget the other, so Low.

RECOMMENDATION

Revoke the old registrar from the deployer account, which holds `ROLE_REGISTRAR_ADMIN` and `ROLE_RENEW_ADMIN` at root.

<https://github.com/SB-Security/audit-2026-09-HoodDomains/blob/master/hood-contracts/contracts/src/access-control/EnhancedAccessControl.sol#L160-L167>

```
hoodRegistry.revokeRootRoles(
  RegistryRolesLib.ROLE_REGISTRAR | RegistryRolesLib.ROLE_RENEW,
```

L-03 · CONTINUED

```
0x59ac59d6a06689cc83b68dab07261cb25b8ca03e  
);
```

Names registered through the old registrar live in the registry and are not affected. Commitments pending on the old registrar cannot be completed there, so make sure the frontend only uses the live registrar before revoking. Do the same for `BatchRegistrar` once pre-migration reservations are no longer needed.

Add the revocation of the previous registrar to the migration script, and record `HoodRegistrarBatch` under `deployments/` with its constructor arguments and deploy transaction.

L-04

LOW

● ACKNOWLEDGED

Deployer key holds every root admin role on HoodRegistry

HoodRegistry is deployed with the deployer as the first root role holder and HOOD_REGISTRY_ROOT as its bitmap.

https://github.com/SB-Security/audit-2026-09-HoodDomains/blob/master/hood-contracts/contracts/deploy/01_HoodRegistry.ts#L23-L27

```
const hoodRegistry = await deploy("HoodRegistry", {
  account: deployer,
  artifact: artifacts.HoodRegistry,
  args: [labelStore.address, deployer, DEPLOYMENT_ROLES.HOOD_REGISTRY_ROOT],
});
```

<https://github.com/SB-Security/audit-2026-09-HoodDomains/blob/master/hood-contracts/contracts/script/deploy-constants.ts#L88-L98>

```
HOOD_REGISTRY_ROOT:
  ROLES.ADMIN.REGISTRY.REGISTRAR |
  ROLES.ADMIN.REGISTRY.REGISTER_RESERVED |
  ROLES.REGISTRY.SET_PARENT |
  ROLES.ADMIN.REGISTRY.SET_PARENT |
  ROLES.ADMIN.REGISTRY.RENEW |
  ROLES.REGISTRY.CAN_NAME |
  ROLES.ADMIN.REGISTRY.CAN_NAME |
  ROLES.REGISTRY.SET_URI |
  ROLES.ADMIN.REGISTRY.SET_URI,
```

Checked on Robinhood Chain on 2026-09-11: the deployer 0xb47821615f96d800cc5c95e436ce66f891d6de3f still holds this full bitmap on the live registry. The documented owner and beneficiary 0x28669d502b930801ac91eda33f56800bf76ce22f holds only ROLE_CAN_NAME. The deployer also owns BatchRegistrar. No root role was ever revoked.

IMPACT

The deployer key, not the documented owner, controls the registry. It can grant itself ROLE_REGISTRAR and register or renew any name for free, grant ROLE_REGISTER_RESERVED, change the parent and replace the URI renderer through setURI.

The scope describes one trusted owner key. In practice there are two keys with different powers, and the stronger one is a deployment key. This needs that key to act or leak, so Low.

RECOMMENDATION

Decide which account should administer the registry, ideally a multisig, and move the roles there. Grant first, then revoke, so no role is ever left without a holder.

<https://github.com/SB-Security/audit-2026-09-HoodDomains/blob/master/hood-contracts/contracts/src/access-control/EnhancedAccessControl.sol#L134-L167>

```
// sent from the deployer
hoodRegistry.grantRootRoles(HOOD_REGISTRY_ROOT, newAdmin);
hoodRegistry.revokeRootRoles(HOOD_REGISTRY_ROOT, deployer);
```

Transfer ownership of BatchRegistrar to the same account. If the deployer key is meant to stay, document it as a trusted key next to the owner.

L-05

LOW

● FIXED

Reserved names cannot be claimed through the live registrars

`BatchRegistrar.batchRegister` reserves a name by registering it with `owner = address(0)`.

<https://github.com/SB-Security/audit-2026-09-HoodDomains/blob/master/hood-contracts/contracts/src/registrar/BatchRegistrar.sol#L61-L71>

```
if (state.status == IPermissionedRegistry.Status.AVAILABLE) {
    REGISTRY.register(labels[i], address(0), registry, resolver, 0, expires[i]);
} else if (
    state.status == IPermissionedRegistry.Status.RESERVED && expires[i] > state.expiry
) {
    REGISTRY.renew(state.tokenId, expires[i]);
}
```

Turning a `RESERVED` name into a `REGISTERED` one needs `ROLE_REGISTER_RESERVED` on the registry root.

<https://github.com/SB-Security/audit-2026-09-HoodDomains/blob/master/hood-contracts/contracts/src/registry/PermissionedRegistry.sol#L453-L466>

```
} else {
    if (prevOwner != address(0)) {
        revert LabelAlreadyRegistered(label);
    } else if (owner == address(0)) {
        revert LabelAlreadyReserved(label);
    }
    if (checkRoles) {
        _checkRoles(ROOT_RESOURCE, RegistryRolesLib.ROLE_REGISTER_RESERVED, msg.sender);
    }
}
```

Neither registrar can do it. Both go through `_requireAvailable`, which only accepts `AVAILABLE` names, and the deploy constants give the registrars `REGISTRAR` | `RENEW` only.

<https://github.com/SB-Security/audit-2026-09-HoodDomains/blob/master/hood-contracts/contracts/src/registrar/HoodRegistrar.sol#L301-L309>

```
return
    state.status == IPermissionedRegistry.Status.AVAILABLE &&
    (grace == (block.timestamp - state.expiry) < GRACE_PERIOD);
```

<https://github.com/SB-Security/audit-2026-09-HoodDomains/blob/master/hood-contracts/contracts/script/deploy-constants.ts#L99-L100>

```
// HoodRegistrar and BatchRegistrar are granted REGISTRAR and RENEW on HoodRegistry root at
static deploy.
HOOD_REGISTRAR_ROOT: ROLES.REGISTRY.REGISTRAR | ROLES.REGISTRY.RENEW,
```

The deployer holds the admin variant of `ROLE_REGISTER_RESERVED` but not the role itself.

IMPACT

A reserved name can only reach its intended owner through a manual step: grant `ROLE_REGISTER_RESERVED` to an account and call `HoodRegistry.register` directly. If that does not happen before `expires[i]`, the name becomes `AVAILABLE` and anyone can register it.

The live registry has no `LabelReserved` event, so no name is affected today. The issue needs the trusted owner to reserve a name and then miss the deadline, so Low.

RECOMMENDATION

If reservations will be used, wire the claim path before reserving anything:

1. Grant `ROLE_REGISTER_RESERVED` at root to the account that will complete reservations, from the deployer.
2. Complete a reservation with a direct `register` call on `HoodRegistry`, passing the intended owner, `REGISTRATION_ROLE_BITMAP` as the role bitmap, and `0` as `expiry` so the reservation's expiry is kept.
3. Pick reservation expiries with enough margin for that step.

If reservations are not planned, revoke the root roles of `BatchRegistrar` instead, as recommended for the superseded registrar in L-03.

L-06

LOW

● ACKNOWLEDGED

A rejecting owner contract blocks role changes on its own name

Every `grantRoles` or `revokeRoles` on a name burns and re-mints the token to the same owner in `_regenerate`.

<https://github.com/SB-Security/audit-2026-09-HoodDomains/blob/master/hood-contracts/contracts/src/registry/PermissionedRegistry.sol#L548-L560>

```
function _regenerate(uint256 resource) internal {
    if (resource != ROOT_RESOURCE) {
        Entry storage entry = _entry(resource);
        uint256 tokenId = _constructTokenId(resource, entry);
        address owner = super.ownerOf(tokenId); // grant/revoke only on registered
        _burn(owner, tokenId, 1);
        ++entry.tokenVersionId;
        uint256 newTokenId = _constructTokenId(tokenId, entry);
        emit TokenRegenerated(tokenId, newTokenId); // resource is unchanged
        _mint(owner, newTokenId, 1, "");
        _broadcastTokenURI(newTokenId);
    }
}
```

`_mint` runs the ERC-1155 acceptance check, so a contract owner whose `onERC1155Received` reverts makes the role change revert.

<https://github.com/SB-Security/audit-2026-09-HoodDomains/blob/master/hood-contracts/contracts/src/erc1155/ERC1155Singleton.sol#L313-L319>

```
function _mint(address to, uint256 id, uint256 value, bytes memory data) internal {
    if (to == address(0)) {
        revert ERC1155InvalidReceiver(address(0));
    }
    (uint256[] memory ids, uint256[] memory values) = _asSingletonArrays(id, value);
    _updateWithAcceptanceCheck(address(0), to, ids, values, data, false);
}
```

IMPACT

A name held by a contract that later stops accepting ERC-1155 tokens (a paused module, an upgraded wallet) can no longer have its roles changed, by the owner or by anyone else. Ownership, resolution and transfers keep working, and the owner can move the name to another account, so only that owner is affected. Low.

RECOMMENDATION

For the next registry version, re-mint to the existing owner without the acceptance check. The owner already accepted the token once, and the re-mint is internal bookkeeping, so skipping the hook adds no new risk. For example, add an internal mint variant in `ERC1155Singleton` that calls `_update` directly and use it in `_regenerate`.

For the live deployment, holders using smart contract wallets should keep the hook accepting, or move the name before changing their wallet.

● INFORMATIONAL SEVERITY · 3 FINDINGS

I-01

INFORMATIONAL

● FIXED

Use assembly call to send native tokens when return data is not needed

The statement `(bool ok, ) = recipient.call{value: amount}("")` forwards ETH and discard the return data. The high-level `.call` path always copies any returndata into memory before discarding it, wasting gas on the copy and exposing the caller to a return-bomb DoS where the recipient returns a huge payload to inflate caller memory expansion costs. Replace with assembly `call(gas(), to, value, 0, 0, 0, 0)` so the EVM never copies returndata.

```
// AbstractHoodRegistrar.sol:L124-136

    if (amount > 0) {
        (bool paid, ) = BENEFICIARY.call{value: amount}("");
        if (!paid) {
            revert PaymentTransferFailed();
        }
    }
    uint256 refund = msg.value - amount;
    if (refund > 0) {
        (bool refunded, ) = msg.sender.call{value: refund}("");
        if (!refunded) {
            revert RefundFailed();
        }
    }
}
```

IMPACT

This wastes gas on the memory allocation and copy, and also exposes the caller to a "return-bomb" DoS where the callee returns a huge payload to grief the caller's gas.

RECOMMENDATION

Use inline assembly to avoid the returndata copy:

```
    bool success;
    assembly {
        success := call(gas(), to, amount, 0, 0, 0, 0)
    }
    if (!success) revert PaymentTransferFailed();
```

I-02

INFORMATIONAL

● FIXED

Missing Validation for Premium Period Parameters

The `EthRentPriceOracle` constructor assigns `premiumHalvingPeriod` and `premiumPeriod` without verifying they are positive. When either is zero, `LibHalving.halving()` short-circuits to zero, silently disabling the premium mechanism that discourages sniping of recently expired names.

```
// EthRentPriceOracle.sol

PREMIUM_PRICE_INITIAL = premiumPriceInitial;
PREMIUM_HALVING_PERIOD = premiumHalvingPeriod;
PREMIUM_PERIOD = premiumPeriod;

// LibHalving.sol

function halving(uint256 initial, uint256 half, uint256 elapsed)
    internal
    pure
    returns (uint256)
{
    if (initial == 0 || half == 0)
        return 0;
```

If `premiumHalvingPeriod == 0`, the `halving()` function returns 0 immediately. This makes `PREMIUM_PRICE_OFFSET` equal 0, and `getPremiumPriceAfter()` returns 0 for every duration:

```
// EthRentPriceOracle.sol

function getPremiumPriceAfter(uint64 duration) public view returns (uint256) {
    return
        duration < PREMIUM_PERIOD
            ? LibHalving.halving(PREMIUM_PRICE_INITIAL, PREMIUM_HALVING_PERIOD, duration) -
              PREMIUM_PRICE_OFFSET
            : 0;
}
```

IMPACT

With a zero halving period, any recently expired name can be re-registered at base price with no premium. The premium exists to deter sniping - where an attacker watches for an expiry, then front-runs the original owner's renewal with a new registration. Disabling it removes that economic deterrent.

This does not affect funds or authorization, only the incentive structure. The current deployment uses `premiumHalvingPeriod = 86400` (1 day) and `premiumPeriod = 1814400` (21 days), so the deployed instance is not affected.

RECOMMENDATION

Add validation in the constructor:

```
if (premiumHalvingPeriod == 0 || premiumPeriod == 0) {
    revert InvalidPremiumConfig();
}
```

I-03

INFORMATIONAL

● ACKNOWLEDGED

Approved ERC-1155 operator can revoke the owner's admin roles and permanently lock the name

`PermissionedRegistry._getRoles` merges the owner's full role bitmap, admin bits included, into any operator the owner approved with `setApprovalForAll`.

<https://github.com/SB-Security/audit-2026-09-HoodDomains/blob/master/hood-contracts/contracts/src/registry/PermissionedRegistry.sol#L596-L610>

```
function _getRoles(uint256 resource, address account) internal view virtual override returns (uint256 roleBitmap) {
    roleBitmap = super._getRoles(resource, account);
    if (resource != ROOT_RESOURCE) {
        address owner = getOwner(resource);
        if (owner != address(0) && owner != account && isApprovedForAll(owner, account)) {
            roleBitmap |= super._getRoles(resource, owner);
        }
    }
}
```

The grant path is protected: `_getSettableRoles` drops admin bits for every token resource.

<https://github.com/SB-Security/audit-2026-09-HoodDomains/blob/master/hood-contracts/contracts/src/registry/PermissionedRegistry.sol#L577-L589>

```
uint256 roleBitmap = super._getSettableRoles(resource, account);
return resource == ROOT_RESOURCE ? roleBitmap : roleBitmap >> 128;
```

The revoke path is not. `_getRevokableRoles` is never overridden, so the base version runs and turns every admin bit the operator inherited into revocation power.

<https://github.com/SB-Security/audit-2026-09-HoodDomains/blob/master/hood-contracts/contracts/src/access-control/EnhancedAccessControl.sol#L434-L441>

```
function _getRevokableRoles(uint256 resource, address account) internal view virtual returns (uint256) {
    return EACBaseRolesLib.withAdminRolesApplied(_effectiveRoles(resource, account));
}
```

Every registration grants the owner `ROLE_CAN_TRANSFER_ADMIN`, `ROLE_SET_RESOLVER_ADMIN` and `ROLE_SET_SUBREGISTRY_ADMIN`.

<https://github.com/SB-Security/audit-2026-09-HoodDomains/blob/master/hood-contracts/contracts/src/registrars/HoodRegistrar.sol#L15-L21>

```
uint256 constant REGISTRATION_ROLE_BITMAP =
    RegistryRolesLib.ROLE_SET_SUBREGISTRY |
    RegistryRolesLib.ROLE_SET_SUBREGISTRY_ADMIN |
    RegistryRolesLib.ROLE_SET_RESOLVER |
    RegistryRolesLib.ROLE_SET_RESOLVER_ADMIN |
    RegistryRolesLib.ROLE_CAN_TRANSFER_ADMIN;
```

`_update` refuses any transfer where the owner lacks `ROLE_CAN_TRANSFER_ADMIN`, and admin bits can never be granted again on a token.

<https://github.com/SB-Security/audit-2026-09-HoodDomains/blob/master/hood-contracts/contracts/src/registry/PermissionedRegistry.sol#L508-L512>

I-03 · CONTINUED

```

    revert TransferDisallowed(tokenId, from);
}

```

So an approved operator can call `revokeRoles` on the owner's own name and strip the owner's admin bits. The code comes from ENSv2 `main` after the audited commit.

IMPACT

After the operator revokes `ROLE_CAN_TRANSFER_ADMIN`, the owner keeps the token but can never transfer it again, and nobody can restore the role. Revoking the other admin bits leaves the owner unable to manage delegated roles, while roles the operator granted to itself survive the end of the approval.

The attacker must first be approved by the owner, and an approved operator can already transfer the name away. The added harm is a permanent, silent lock instead of theft. ENSv2 treats an approved operator as an alias of the owner and ships a test for this revocation, so it is an inherited design choice. Informational.

RECOMMENDATION

The live HoodRegistry is not upgradeable, so no change can reach the deployed registry. Two options:

Option 1, accept and document (no contract change). Treat operator approval as full control of the name, as ENSv2 intends. State in the user docs that approving an operator also lets it burn the owner's admin roles, including `ROLE_CAN_TRANSFER_ADMIN`, which makes the name soulbound. Have the frontend show that warning before `setApprovalForAll`, and advise users to approve only audited marketplace contracts.

Option 2, fix in the next registry deployment. Add the `_getRevokableRoles` override below to `PermissionedRegistry`. Operators keep the alias for regular roles and lose only the ability to revoke admin roles through inherited bits. Owner and root powers are unchanged. Update the inherited `test_setApprovalForAll_revokeRoles` to expect `EACCannotRevokeRoles`. Verified on this codebase: 317 of 319 tests pass, the remaining failure is an unrelated pre-existing fuzz flake. If ENS adopts the same change upstream, it can be taken from there instead.

<https://github.com/SB-Security/audit-2026-09-HoodDomains/blob/master/hood-contracts/contracts/src/registry/PermissionedRegistry.sol#L591-L610>

```

import {EACBaseRolesLib} from "../access-control/libraries/EACBaseRolesLib.sol";

/// @inheritdoc EnhancedAccessControl
/// @dev An approved operator may revoke regular roles through the owner's admin roles,
///      but may revoke admin roles only through admin roles it holds itself.
///      Admin roles are burned by the owner, e.g. `ROLE_CAN_TRANSFER_ADMIN` to make a name
soulbound.
function _getRevokableRoles(uint256 resource, address account)
    internal
    view
    virtual
    override
    returns (uint256)
{
    uint256 roleBitmap = super._getRevokableRoles(resource, account);
    if (resource != ROOT_RESOURCE) {
        address owner = getOwner(resource);
        if (owner != address(0) && owner != account && isApprovedForAll(owner, account)) {
            uint256 ownRoles = super._getRoles(ROOT_RESOURCE, account) |
super._getRoles(resource, account);
            uint256 ownRevokable = EACBaseRolesLib.withAdminRolesApplied(ownRoles);
            roleBitmap = (roleBitmap & type(uint128).max) | (ownRevokable &
~uint256(type(uint128).max));

```

I-03 • CONTINUED

```
    }  
  }  
  return roleBitmap;  
}
```



AUDIT COMPLETE

Stay secure.

Thank you for choosing SBSecurity.

WEBSITE

sbsecurity.net

TELEGRAM

[@sbsecurity_audits](https://t.me/sbsecurity_audits)

REPORT

Hood Domains · September 2026